

Optimización - Universidad Nacional del Altiplano - FINESI

Evaluación: Unidad II

Apellidos y Nombres: Beatriz Umiña Machaca

Pregunta 1

¿Cuáles son las principales diferencias entre la optimización de funciones convexas y no convexas, y cómo influyen en la elección de métodos?

Respuesta:

Las principales diferencias son:

Funciones Convexas:

- Cualquier mínimo local es también mínimo global
- Garantizan convergencia a la solución óptima
- Métodos eficientes: descenso de gradiente, Newton, quasi-Newton
- Condiciones de optimalidad: $\nabla f(x^*) = 0$ es condición necesaria y suficiente

Funciones No Convexas:

- Múltiples mínimos locales
- No garantizan convergencia al mínimo global
- Requieren métodos más sofisticados: algoritmos genéticos, simulated annealing, métodos de enjambre
- Condiciones de optimalidad: $\nabla f(x^*) = 0$ solo es condición necesaria

Influencia en la elección de métodos: Para funciones convexas se prefieren métodos determinísticos y eficientes, mientras que para no convexas se requieren métodos estocásticos o metaheurísticos que exploren mejor el espacio de búsqueda.

Pregunta 2

¿Cuándo resulta más efectivo usar variantes del descenso de gradiente y cuáles son los factores clave en su selección?

Respuesta:

Variantes y sus casos de uso:

- **Gradient Descent (GD):** Datasets pequeños, funciones suaves
- **Stochastic Gradient Descent (SGD):** Datasets grandes, cuando se requiere velocidad
- **Mini-batch GD:** Balance entre precisión y eficiencia computacional
- **Adam:** Redes neuronales, problemas con gradientes dispersos
- **RMSprop:** Redes neuronales recurrentes
- **Adagrad:** Problemas con características dispersas

Factores clave en la selección:

1. Tamaño del dataset
2. Naturaleza del problema (convexo/no convexo)
3. Recursos computacionales disponibles
4. Velocidad de convergencia requerida
5. Estabilidad del gradiente
6. Características del espacio de parámetros

Pregunta 3

¿Cómo se aplican los criterios de optimalidad en un método de optimización?

Respuesta:

Criterios de Optimalidad:

Condiciones de primer orden (necesarias):

$$\nabla f(x^*) = 0$$

Condiciones de segundo orden (suficientes):

$$\nabla^2 f(x^*) \succ 0 \text{ (definida positiva)}$$

Aplicación en métodos de optimización:

1. **Criterio de parada:** $\|\nabla f(x_k)\| < \epsilon$
2. **Dirección de búsqueda:** $d_k = -\nabla f(x_k)$
3. **Verificación de convergencia:** Evaluar si se cumple $\nabla f(x_k) \approx 0$
4. **Test de optimalidad:** Verificar que la matriz Hessiana sea definida positiva

Con restricciones (Condiciones KKT):

$$\nabla f(x^*) + \sum_{i=1}^m \lambda_i \nabla g_i(x^*) = 0 \quad (1)$$

$$g_i(x^*) \leq 0, \quad i = 1, \dots, m \quad (2)$$

$$\lambda_i \geq 0, \quad i = 1, \dots, m \quad (3)$$

$$\lambda_i g_i(x^*) = 0, \quad i = 1, \dots, m \quad (4)$$

Pregunta 4

¿De qué manera facilitan las herramientas de optimización automática (p.ej., Optuna) la búsqueda de configuraciones óptimas en aprendizaje automático?

Respuesta:

Facilidades que proporciona Optuna:

- **Algoritmos de optimización avanzados:** TPE (Tree-structured Parzen Estimator), CMA-ES, búsqueda aleatoria
- **Pruning automático:** Detiene trials no prometedores tempranamente
- **Paralelización:** Ejecución distribuida de múltiples trials
- **Persistencia:** Almacenamiento de estudios y reanudación
- **Visualización:** Gráficos de convergencia y análisis de parámetros

Ventajas en ML:

1. Automatización completa del proceso de búsqueda
2. Manejo eficiente de espacios de hiperparámetros complejos
3. Reducción significativa del tiempo de búsqueda
4. Integración sencilla con frameworks de ML
5. Soporte para optimización multiobjetivo

Pregunta 5

¿Cómo influye la selección de estrategias de regularización en el desempeño y la generalización de un modelo?

Respuesta:

Tipos de regularización y sus efectos:

L1 (Lasso):

$$J(w) = \frac{1}{2m} \sum_{i=1}^m (h_w(x^{(i)}) - y^{(i)})^2 + \lambda \sum_{j=1}^n |w_j|$$

- Produce sparsity (selección de características)
- Útil cuando hay muchas características irrelevantes

L2 (Ridge):

$$J(w) = \frac{1}{2m} \sum_{i=1}^m (h_w(x^{(i)}) - y^{(i)})^2 + \lambda \sum_{j=1}^n w_j^2$$

- Reduce la magnitud de los pesos
- Mejor para multicolinealidad

Elastic Net:

$$J(w) = \frac{1}{2m} \sum_{i=1}^m (h_w(x^{(i)}) - y^{(i)})^2 + \lambda_1 \sum_{j=1}^n |w_j| + \lambda_2 \sum_{j=1}^n w_j^2$$

Dropout:

- Desactiva aleatoriamente neuronas durante el entrenamiento
- Previene co-adaptación de características

Impacto en el desempeño:

- Reduce overfitting
- Mejora la generalización
- Puede reducir la capacidad del modelo si se aplica excesivamente
- Requiere ajuste cuidadoso del parámetro de regularización λ

Pregunta 6

¿Cómo se pueden integrar rutinas de validación y corrección de código en Optuna para mejorar la calidad y eficacia del proceso de optimización de hiperparámetros?

Respuesta:

Estrategias de integración:

1. Validación cruzada dentro del objetivo:

- Usar k-fold cross-validation en cada trial
- Reportar métricas estadísticamente robustas

2. Manejo de excepciones:

- Try-catch blocks para capturar errores
- Retornar valores predeterminados en caso de fallo

3. Callbacks personalizados:

- Validación de parámetros antes de entrenar
- Logging detallado de errores

4. Pruning inteligente:

- Usar MedianPruner o PercentilePruner
- Configurar early stopping basado en métricas de validación

Ejemplo de implementación:

```
def objective(trial):
    try:
        # Validación de parámetros
        params = {
            'learning_rate': trial.suggest_float('lr', 1e-5, 1e-1, log=True),
            'batch_size': trial.suggest_categorical('batch_size', [16, 32, 64])
        }

        # Validación cruzada
        cv_scores = cross_val_score(model, X, y, cv=5)

        # Reporte de métricas intermedias
        for i, score in enumerate(cv_scores):
            trial.report(score, i)
            if trial.should_prune():
                raise optuna.TrialPruned()

        return cv_scores.mean()

    except Exception as e:
        logging.error(f"Error in trial {trial.number}: {e}")
        return float('inf') # Penalizar trials con errores
```

Pregunta 7

¿Cómo podemos utilizar las métricas provistas por scikit-learn para evaluar un modelo de clasificación, y qué información adicional brindan además de la exactitud?

Respuesta:

Código completado:

```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

y_true = [0, 1, 2, 2, 2]
y_pred = [0, 2, 2, 2, 1]

print("Accuracy:", accuracy_score(y_true, y_pred))
print("Precision:", precision_score(y_true, y_pred, average='macro'))
print("Recall:", recall_score(y_true, y_pred, average='macro'))
print("F1-score:", f1_score(y_true, y_pred, average='macro'))
print("Classification Report:")
print(classification_report(y_true, y_pred))
print("Confusion Matrix:")
print(confusion_matrix(y_true, y_pred))
```

Información adicional de las métricas:

- **Precision:** $\frac{TP}{TP+FP}$ - Proporción de predicciones positivas correctas
- **Recall (Sensibilidad):** $\frac{TP}{TP+FN}$ - Proporción de casos positivos correctamente identificados
- **F1-score:** $2 \cdot \frac{Precision \cdot Recall}{Precision + Recall}$ - Media armónica entre precisión y recall
- **Specificity:** $\frac{TN}{TN+FP}$ - Proporción de casos negativos correctamente identificados

Ventajas sobre accuracy:

1. Mejor evaluación en datasets desbalanceados
2. Información específica por clase
3. Identificación de sesgos del modelo
4. Comprensión del tipo de errores cometidos

Interpretación de resultados: Con los datos dados:

- Accuracy = 0.6 (60 % de predicciones correctas)
- La precision y recall por clase revelan el desempeño específico
- La matriz de confusión muestra exactamente qué clases se confunden